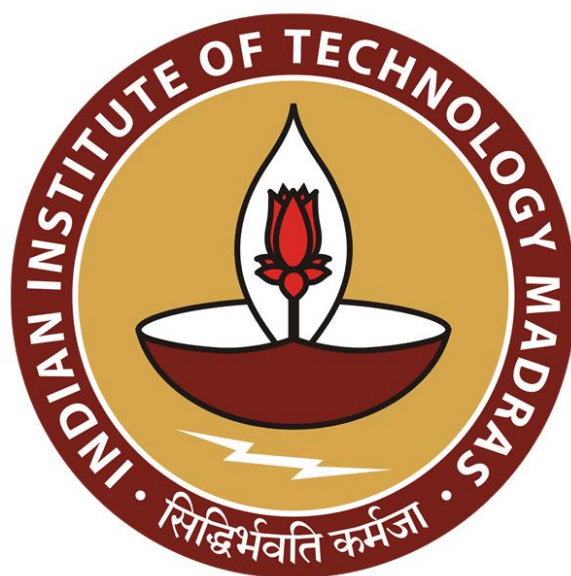




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Graphical Interfaces and Event-Driven Programming

(Refer Slide Time: 0:15)



Graphical interfaces and event-driven programming

Madhavan Mukund
<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java
Week 12



For the last item in the course, we will look at graphical interfaces. So, how do we program windows and buttons and these kinds of things that we see when we interact on a menu with the user interface, and how this is connected to what is called event driven programming.

(Refer Slide Time: 0:32)



GUIs and events

- How do we design graphical user interfaces?
- Multiple applications simultaneously displayed on screen
- Keystrokes, mouse clicks have to be sent to appropriate window
- In parallel to main activity, record and respond to these **events**
 - Web browser renders current page
 - Clicking on a link loads a different page



Madhavan Mukund Graphical interfaces and event-driven programming Programming Concepts using Java 2 / 8

So, let us look at graphical user interfaces. So, for the last several years now several decades, probably it has become a standard thing that when we interact with a computer, we use a graphical interface. So, we open an application, it opens up a window, we click various things on it, and various things happen. So, the traditional interface for a computer is a command line where you type the commands explicitly and you execute them according to that. But this is much more user friendly. And this is what most casual users of computers are now used to.

So, when we have a graphical user interface, as we know, multiple things are displayed on the screen. So, at the moment, on my screen, I am seeing one application, but even this application, many things, so I can, for example, I can write with a pen, or I can go forward and backward using the mouse. Or I can go on the left and click on any of these icons here and do something. So, there are many things that can be done. And the question is, how to interact or how to interact with the user in this context.

So, when I type something, or when I click on the mouse, or when I write on the screen, my pen, first of all, it has to be understood which part of the graphical interface I am interacting with. So, that is first thing that is to map what I am doing on the screen, with what is actually running in the background. So, which part of the screen has, for example, if I write something which part of the screen should it display that writing on, so wherever I touch the screen, or if I click on a button, which button have I clicked on.

And of course, once you understand which application component you are interacting with, then you have to actually respond to it. So, this is obviously something which is happening in parallel. So, we talked about threads. So, this can only be done in a threaded mode, where we have essentially an user interaction thread where the user is interacting with your application by clicking on the mouse or writing on the screen or typing or something. And in parallel, something else is reacting to these events.

So, these are these events, which come from the user, and to which the program has to react by doing the appropriate thing. For instance, if I say Refresh, the web browser must refresh the page. And if I click on, if I type in a URL and click load, then it must load a new page and so on.

(Refer Slide Time: 2:52)



Keeping track of events

- Remember coordinates and extent of each window
- Track coordinates of mouse
- OS reports mouse click at (x, y)
 - Check which windows are positioned at (x, y)
 - Check if one of them is "active"
 - Inform that window about mouse click
- Tedious and error-prone
- Programming language support for higher level events
 - Run time support for language maps low level events to high level events
 - OS reports low level events: mouse clicked at (x, y) , key 'a' pressed
 - Program sees high level events: Button was clicked, box was ticked ...



Madhavan Mukund

Graphical interfaces and event-driven programming

Programming Concepts using Java 3 / 8

So, how do we do this? So, how do we keep track of this connection between what the user is doing with the interface and what is happening in the interface, so which part of the application that is on display on the screen, and there may be also multiple applications, so which application is being displayed, so is the click pertaining to my application at all. And if so, which part of my application is the person clicking? So, you could do this, obviously, by remembering at every point, the exact location of the display related to the application, so you could remember the coordinates, the position, and the size, the extent of every window that is open on the screen.

And then you can also track where the mouse is or where the keyboard is typing and so on. And then after this, if the operating system reports that the mouse was clicked at a particular position (x, y) , so measured with respect to the screen, then you then check which windows are currently at that position. And then you see among them which one is active, because you could have one hidden behind the other. And then finally, whichever one is active has to be informed that the mouse was clicked.

So, this is obviously tedious and error-prone. Now more than informing the application that the mouse was clicked as (x, y) , then the application also know has to know at (x, y) , which button in the application was there at that position. So, that is an extra level of complication with respect to even knowing whether it was clicked but what was clicked. So, this as you can imagine, is very tedious. And you could also understand that it is error-prone because you

have to keep track of all these things and do some calculations and you could very well get it wrong.

So, what we are interested in is how a programming language would provide support for these kinds of events. So, these events are things like writing on the screen or clicking a mouse or typing something the keyboard between these events and how they are managed. So, that the programmer of these kind of interfaces has an easier task of explaining in the code what is to be done when something happens and does not have to manually keep track of all these very tedious details.

So, you would like some runtime support at the very least runtime support should map these lower level events to higher level events, as we said. So, basically, clicking on a mouse at position (x, y) should be translated to click this OK button in this window. So, that is a higher level event. So, the OS will report the lower level events, what was clicked, what was pressed, and the program will understand from that automatically, the higher level events, so this button was clicked, or that box was ticked, and so on.

So, this would be a first step. So, you want to have at least this level of detail, taken away from the programmer's viewpoint. So, a programmer does not have to know, really where exactly the mouse was clicked, in terms of the position on the screen, but rather needs to know which part of the code was clicked. So, this is still a little low level. So, what we will see is something even better than this.

(Refer Slide Time: 6:01)

- Programmer directly defines **components** such as windows, buttons, ... that "generate" high level events
- Each event is associated with a **listener** that knows what to do
 - e.g., clicking **Close window** exits application
- Programming language has mechanisms for
 - Describing what types of events a component can generate
 - Setting up an association between components and listeners
- Different events invoke different functions
 - Window frame has **Maximize, Iconify, Close** buttons
- Language "sorts" out events and automatically calls the correct function in the listener



Madhavan Mukund

Graphical interfaces and event-driven programming

Programming Concepts using Java 4/8

So, if we want better programming language support, then what we would like is that the programmer directly reasons about the kind of components that you use when you design these graphical interfaces. So, you define a window, and then you define in that window, certain buttons that can be clicked, you have a certain part of the window that is displaying some information that may be updated, it may be taking that information, if it is a web browser, it might be taking it from an external HTML file, or if it is a, say, a document viewer, like a PDF viewer, it might be taking the display from a file on the disk, and then re-displaying it each time it is asked to do so.

So, you have these windows buttons. And these buttons now change somehow the behavior of this component, but this component itself is a high level component like a window. So, the way we set up this connection, is by saying, if this kind of an event happens, then call this function. And this function is delegated to something called a listener. So, for instance, most applications will have an X somewhere on the top right, where if you click that X, the window closes and the application quits.

So, there is no direct connection between clicking the X and the application closing. So, there will be a similar button, which allows you to minimize the window. So, just clicking on a some point at the corner of the window does not mean that something specific is going to happen. It is the interpretation of that which is decided by the system and the programmer and the interface that is explained to the user. So, the user is told, if you click on the

underscore button, typically it will minimize, if you click on the square button, it will go to full screen, if you click on the X button, it will close.

So, what we would like to do is program at that level. So, we would like to say that there are these 3 buttons. And if the if you detect a click on this type of button, do this, on that type of button, do that and so on. So, the programming language should now be able to keep track of this. It should be able to tell you, if I have a window, what kind of events that window can generate, and then set up a connection saying if this type of event happens, then do this kind of perform this kind of function.

So, this is what is called a listener, somebody is listening is waiting for things to happen. And when something happens, they react and respond. So, obviously, these events differ from component to component, so we discussed in a window, a typical window, the application window, you could have a maximize, which is usually drawn as a square box, or an icon if I minimize which is usually written as an underscore, or a close, which is usually written as an X. So, in our language, we should be able to describe all of these things abstractly, and the system should tell us immediately which event happened.

(Refer Slide Time: 8:47)

An example

- A **Button** with one event, press button
- Pressing the button invokes the function `buttonpush(..)` in a listener

```
interface ButtonListener{
    public abstract void buttonpush(...);
}

class MyClass implements ButtonListener{
    ...
    public void buttonpush(...){
        ...
        // what to do when
        // a button is pushed
    }
}
```

Madhavan Mukund | Graphical interfaces and event-driven programming | Programming Concepts using Java | 5/8

So, let us look at a very simple example of how this could be done. So, we just have something very basic, which is a button that we can press on the screen. So, we use the mouse or we use a stylus or whatever, and we go and we position our cursor at that mouse button and we press it. So, what will this do? Well, this will send a signal to whichever

application it was that the button was pressed. So, effectively, there is only one thing that a button can be do. We can do with a button which is to push it. So, there will be a function which is called when the button is pushed.

So, let us assume that it is called button push. So, this will happen now, in indirectly or without us doing anything actively whenever we as the user, press the button on the screen internally the program will transfer control to a button push function. So, this button push function will be described in some interface and it will be implemented by some object. So, this object will know what to do when button push is called. So, it will invoke certain things. So, in some cases, as we said the button push might so each of these is a button.

For example, Iconify or maximize or close the window is a button. So, depending on which button it is, it will either kill the application or it will magnify the view to the full screen and so on.

(Refer Slide Time: 10:11)

An example

- A `Button` with one event, press button
- Pressing the button invokes the function `buttonpush(..)` in a listener
- We have set up an association between `Button b` and a listener `ButtonListener m`

```
interface ButtonListener{
    public abstract void buttonpush(...);
}

class MyClass implements ButtonListener{
    ...
    public void buttonpush(...){
        ...      // what to do when
                // a button is pushed
    }
}

Button b = new Button();
MyClass m = new MyClass();
b.add_listener(m); // Tell b to notify
                  // m when pushed
```

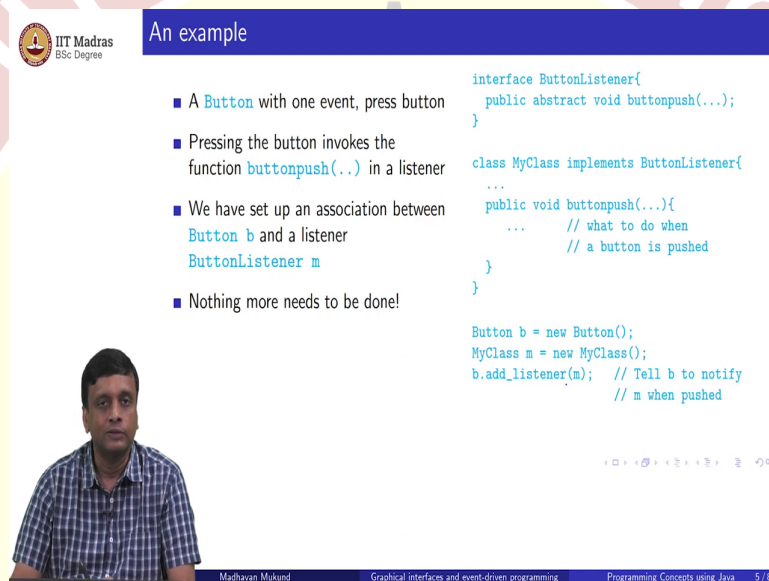
Madhavan Mukund | Graphical interfaces and event-driven programming | Programming Concepts using Java | 5 / 8

So, we of course have to connect each button to the function that performs the correct thing. So, here is a very simple thing. So, we create a button then we create a listener class object which can analyze, button pushes and do one kind of response. And then we say this button. So, for example, if this, so, supposing we have 3 types of buttons, so we could have, as we said, a minimize button, full screen button and a close button. So, we would have 3 different button objects. And we would see 3 different listener objects.

So, the one listener object will know how to Iconify, one listener object will know how to take an application to full screen. And one application, one listener object will know how to close the application gracefully, shut it down. And now for the corresponding button, you have to tell it talk to the corresponding listener. So, that is what happens here.

So, you take this button, and you say, add this listener. So, m is now going to be the object that will respond to button push requests that come from this button, the request itself is going to be done automatically.

(Refer Slide Time: 11:16)



An example

- A `Button` with one event, press button
- Pressing the button invokes the function `buttonpush(...)` in a listener
- We have set up an association between `Button b` and a listener `ButtonListener m`
- Nothing more needs to be done!

```
interface ButtonListener{
    public abstract void buttonpush(...);
}

class MyClass implements ButtonListener{
    ...
    public void buttonpush(...){
        ... // what to do when
        ... // a button is pushed
    }
}

Button b = new Button();
MyClass m = new MyClass();
b.add_listener(m); // Tell b to notify
                  // m when pushed
```

Madhavan Mukund | Graphical interfaces and event-driven programming | Programming Concepts using Java | 5/8

So, we do not have to do anything more than this. We do not have to say if the button is pushed called that, because that is what the programming language gives us for free.

(Refer Slide Time: 11:24)

An example ...

- Communicating each button push to the listener is done automatically by the run-time system
- Information about the button push event is passed as an object to the listener
- `buttonpush(...)` has arguments
 - Listener can decipher source of event, for instance

```
interface ButtonListener{
    public abstract void buttonpush(...);
}

class MyClass implements ButtonListener{
    ...
    public void buttonpush(...){
        ...           // what to do when
                    // a button is pushed
    }
}

Button b = new Button();
MyClass m = new MyClass();
b.add_listener(m);    // Tell b to notify
                    // m when pushed
```



Madhavan Mukund

Graphical interfaces and event-driven programming

Programming Concepts using Java 6/8

So, we just define these two endpoints, we say here is a button. And a button is a, an object, which has certain characteristics, one of which is that if it is pressed by the user, then button push is called. And on the other side, we have a listener, which has an implementation a button push, and we have this connection saying that this button listens is talks to this listener. So, this listener is waiting for button pushes from this event, from this button.

So, therefore, if you have 3 buttons, and 3 listeners, you could have 1 listener per button doing different things, so that the 3 buttons have different interpretations. So, what happens now is that when the button is pushed, the information about the button push is automatically passed to the associated listener. And this button push could have as we will see more information associated with it.

So, therefore, we have in general, it is not just a yes-no, it is not just a Boolean fact that the button was pushed. But a button was pushed and then it might give you some identification about which button was pushed, we will see that you could have more flexible ways of connecting these objects to listen. So, you might want to know or the different button pushes that I could be seeing which one have I pushed, which one am I see.

(Refer Slide Time: 12:38)

- Recall **Timer** example
- **Myclass m** creates a **Timer t** that runs in parallel
- **Timer t** notifies a **Timerowner** when it is done, via a function **timerdone()**
- Abstractly, timer duration elapsing is an **event**, and **Timerowner** is notified when the event occurs
 - In the timer, the notification is done explicitly, manually
 - In the button example, the notification is handled internally, automatically
- In our example, **Myclass m** was itself the **Timerowner** to be notified
- In principle, **Timer t** could be passed a reference to **any** object that implements **Timerowner** interface



Madhavan Mukund

Graphical interfaces and event-driven programming

Programming Concepts using Java 7/8

So, let us connect this back to something that we saw quite some time ago when we talked about callbacks. So, we talked about a timer. And we said that in a timer, we create a class, which in some sense starts a timer in parallel. And when the timer elapses, it should call back my class. So, the timer notifies the class. And we said that because we do not want to make it specific to one class. The notification we will encapsulate in an interface called timer owner and we will fix the function that the timer owner is supposed to use to respond to this timeout.

So, there is a function called timer done, which is called by the timer when its time elapses. And this timer done will be called with respect to the person the class, the object, which created the time. So, abstractly, the timer sleeps for some time and then the time runs out. So, this is an event in the same sense as clicking on a box on the screen or typing something is an event. It is an activity which is undertaken at some point. And when this event happens, a certain response has to be generated and that is what this timer done is doing.

So, what we have done in this example of the timer is we have set up this association between the timer and the class which can then be notified when the timer elapses. So, the difference between the timer class as we have set it up and what we are talking about in the graphical user interface is how this notification happens when the event actually transpires. So, in our graphical user interface, we are saying that the programmer does not have to add any code saying when the button is pressed, call this function. So, that is implicit.

So, there is a function which we know is associated with buttons, called button push, and there is an interface which implements button push. And when the button is pushed, this function is automatically called by the underlying runtime by the Java runtime system. So, the programmer does not have to say if button was pushed then call button push. Whereas in the timer situation, the timer had to explicitly call it so that see only difference that in the timer thing the event was the triggering function was manually called by the object that generated the event.

There is not a graphical user interface. This is done implicitly by the underlying system, that connection between the event happening and the function being invoked. Now, one important thing to remember is that we said that we had my class, and that created a timer. And this timer called back my class. So, my class, the class that created the timer was responsible for responding to the timeout. But there is absolutely no reason why this should be the situation, I could say that my class creates a timer and tells it that there is another object in prime, which will do the job when your time runs out.

So, I could create a timer with a timeout and say, when you are done notify the other object. And this is possible even in our graphical user interface, as we will see. So, the point is that there is a more abstract notion of events and event driven programming. So, this kind of callback kind of situation is an example of that. So, you have something and then when a situation occurs, at that time, some other function is invoked. So, the other function is invoked when the event occurs manually.

But now in our graphical user interface is convenient to adapt this and say that whenever the user interacts with the interface, each interaction constitutes an event and there is an implicit function that is invoked, and we just have to describe what that function should do. So, that we get this association between events and the responses.

(Refer Slide Time: 16:29)

- Event driven programming is a natural way of dealing with graphical user interface interactions
- User interacts with object through mouse clicks etc
- These are automatically translated into events and passed to listeners
- Listeners implement methods that react appropriately to different types of events



Madhavan Mukund

Graphical interfaces and event-driven programming

Programming Concepts using Java 8 / 8

So, in summary, event driven programming is a very natural way of dealing with graphical user interface interaction. So, if we want programming language support for these kinds of applications, which of course we do, because so many applications now require this kind of interaction with the user, then programming in terms of events and responses is a very natural way to do it.

So, the user interacts with the object, it could be a window on the screen or it could be something else using mouse clicks, touch, and so on. And these are automatically translated into events and passed to the listeners and the listeners implement methods that react appropriately to different types of events. So, what we will see next is how this is actually achieved within Java.